

**ANALISIS KOMPARATIF TEKNIK SEGMENTASI CITRA DIGITAL:
THRESHOLDING, DETEKSI TEPI, DAN PENDEKATAN BERBASIS WILAYAH
PENGOLAHAN CITRA DIGITAL**



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Pendahuluan

Segmentasi citra merupakan tahapan fundamental dalam bidang pengolahan citra digital dan visi komputer. Proses ini bertujuan untuk mempartisi citra menjadi beberapa wilayah atau objek yang memiliki karakteristik serupa, seperti intensitas warna atau tekstur. Dalam praktiknya, segmentasi berfungsi untuk memisahkan objek utama (foreground) dari latar belakang (background) guna mempermudah proses analisis dan ekstraksi informasi lebih lanjut. Makalah ini membahas implementasi dan analisis dari berbagai pendekatan segmentasi, mulai dari teknik ambang batas (thresholding) hingga metode berbasis tepi dan wilayah menggunakan pustaka OpenCV dan NumPy.

B. Segmentasi Berbasis Ambang Batas (Thresholding)

Teknik thresholding merupakan metode segmentasi yang paling sederhana namun efektif untuk citra dengan kontras yang jelas. Eksperimen dilakukan terhadap empat metode utama:

1. Global Thresholding

Metode ini menerapkan satu nilai ambang batas statis (dalam kasus ini $T = 127$) untuk seluruh piksel dalam citra. Meskipun efisien secara komputasi, metode ini sangat rentan terhadap kegagalan jika citra memiliki distribusi pencahayaan yang tidak merata atau noise yang signifikan.

2. Metode Otsu

Algoritma Otsu bekerja secara otomatis dengan mencari nilai ambang batas yang meminimalkan variansi dalam kelas (intra-class variance). Teknik ini sangat optimal untuk citra dengan histogram bimodal, di mana terdapat pemisahan yang jelas antara puncak intensitas objek dan latar belakang.

3. Adaptive Thresholding

Berbeda dengan metode global, *adaptive thresholding* menghitung ambang batas untuk setiap area kecil (lokal) pada citra menggunakan Gaussian weighting. Pendekatan ini terbukti paling tangguh dalam menangani citra dengan pencahayaan yang tidak seragam (*uneven illumination*), karena mampu beradaptasi dengan perubahan intensitas cahaya di berbagai bagian citra.

4. Iterative Thresholding

Metode ini menggunakan pendekatan heuristik untuk menemukan ambang batas optimal melalui perhitungan rata-rata intensitas latar depan dan latar belakang secara berulang hingga mencapai konvergensi. Ini memberikan fleksibilitas lebih

baik daripada nilai statis namun memerlukan beban komputasi yang lebih tinggi dibandingkan metode Otsu.

C. Analisis Kinerja dan Metrik Evaluasi

Berdasarkan hasil eksperimen terhadap citra bimodal, citra dengan noise, dan citra dengan pencahayaan tidak merata, dilakukan evaluasi kuantitatif menggunakan beberapa parameter:

- **Accuracy & Precision:** Mengukur ketepatan piksel yang diklasifikasikan sebagai objek.
- **Recall & F1-Score:** Menilai kemampuan algoritma dalam menemukan seluruh bagian objek asli tanpa banyak kesalahan.
- **Intersection over Union (IoU):** Menghitung persentase tumpang tindih antara hasil segmentasi dengan *ground truth*.

Hasil menunjukkan bahwa pada citra ideal (bimodal), Otsu dan Iterative Thresholding menghasilkan skor F1 yang hampir sempurna. Namun, pada citra dengan gradien cahaya, *Adaptive Thresholding* mengungguli metode lainnya dengan margin yang signifikan, membuktikan bahwa satu nilai ambang batas tunggal tidak cukup untuk menangani variasi luminansi kompleks.

D. Deteksi Tepi sebagai Basis Segmentasi

Deteksi tepi bertujuan untuk menangkap perubahan diskontinuitas dalam intensitas citra. Pendekatan ini sangat berguna untuk mengenali struktur geometris objek.

1. Operator Gradien (Sobel dan Prewitt)

Operator Sobel dan Prewitt bekerja dengan menghitung turunan pertama dari intensitas citra. Sobel menggunakan kernel yang memberikan bobot lebih pada piksel pusat, yang memberikan efek penghalusan noise yang lebih baik dibandingkan Prewitt. Kedua metode ini efektif untuk mendeteksi *step edges* namun cenderung menghasilkan garis tepi yang tebal.

2. Algoritma Canny

Canny dianggap sebagai standar emas dalam deteksi tepi karena melibatkan beberapa tahapan: pengurangan noise (Gaussian blur), pencarian gradien, *non-maximum suppression*, dan *hysteresis thresholding*. Hasilnya adalah tepi yang tipis, bersih, dan terhubung dengan baik, sehingga sangat optimal untuk interpretasi bentuk objek.

3. Operator Laplacian

Berbeda dengan Sobel, Laplacian menggunakan turunan kedua (operator linear). Metode ini sangat sensitif terhadap noise namun sangat baik dalam mendeteksi perubahan intensitas yang sangat halus. Dalam implementasi praktikum, Laplacian seringkali memerlukan tahap prapemrosesan berupa *blurring* untuk menghindari deteksi tepi palsu akibat noise.

E. Segmentasi Berbasis Wilayah (Region-Based)

Pendekatan ini tidak mencari perbedaan (tepi), melainkan mencari kesamaan antar piksel untuk membentuk suatu wilayah.

1. Region Growing

Metode ini dimulai dari titik benih (*seed points*) yang ditentukan secara manual atau otomatis. Algoritma kemudian memeriksa piksel tetangga dan menggabungkannya ke dalam wilayah jika selisih intensitasnya berada di bawah ambang batas tertentu. Keunggulan utama metode ini adalah kemampuannya menghasilkan wilayah yang terhubung sempurna dan bersih dari noise luar, asalkan benih diletakkan dengan tepat pada objek target.

2. Split and Merge

Teknik ini menggunakan pendekatan hierarki (serupa dengan struktur data *quadtree*). Citra akan terus dipecah (*split*) menjadi kuadran-kuadran kecil jika wilayah tersebut tidak homogen. Sebaliknya, wilayah-wilayah kecil yang memiliki karakteristik serupa akan digabungkan (*merge*). Dalam implementasi ini, metode ini digunakan untuk menyederhanakan representasi citra menjadi blok-blok intensitas yang konsisten.

F. Kesimpulan dan Rekomendasi

Dari seluruh rangkaian pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan strategis:

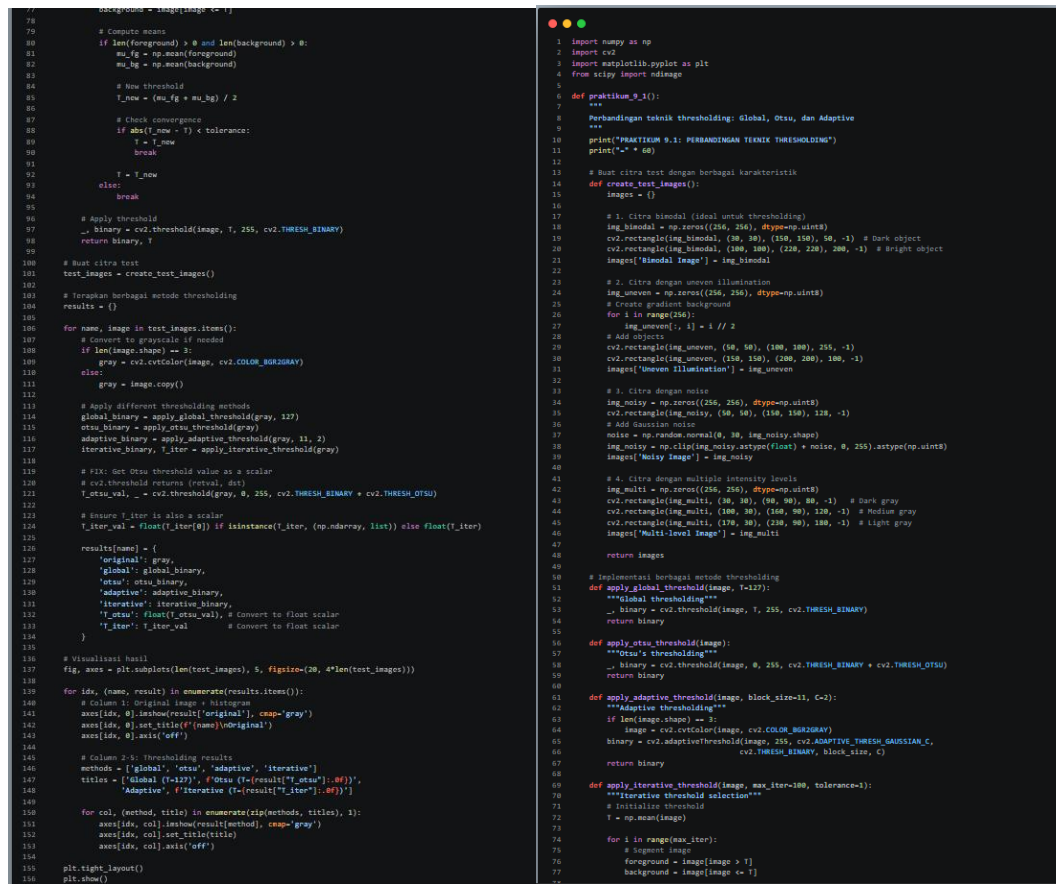
1. **Pemilihan Metode:** Tidak ada satu metode tunggal yang superior untuk semua jenis citra. **Otsu** adalah pilihan terbaik untuk citra standar dengan kontras tinggi karena kecepatannya. **Adaptive Thresholding** wajib digunakan untuk citra dengan pencahayaan dinamis.
2. **Ketangguhan Noise:** Deteksi tepi **Canny** menunjukkan performa paling stabil terhadap gangguan noise dibandingkan operator gradien sederhana lainnya.

3. **Akurasi Geometris:** Segmentasi berbasis wilayah seperti **Region Growing** memberikan hasil yang lebih intuitif untuk pemisahan objek secara utuh dibandingkan hanya mengandalkan garis tepi yang terkadang terputus.

Implementasi kode ini memberikan fondasi yang kuat bagi pengembangan sistem visi komputer yang lebih kompleks, seperti pengenalan objek medis atau sistem navigasi robotika, di mana kualitas segmentasi awal sangat menentukan akurasi hasil akhir.

G. Lampiran Program dan Screenshot Output

1. Screenshot Program



```

77 background = image[image < 0]
78
79 # Compute means
80 if len(background) > 0 and len(background) > 0:
81     mu_fg = np.mean(background)
82     mu_bg = np.mean(background)
83
84     # New threshold
85     T_new = (mu_fg + mu_bg) / 2
86
87     # Check convergence
88     if abs(T_new - T) < tolerance:
89         T = T_new
90         break
91     else:
92         T = T_new
93         break
94
95 # Apply threshold
96 _, binary = cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
97
98 return binary, T
99
100 # Buat citra test
101 test_images = create_test_images()
102
103 # Terapkan berbagai metode thresholding
104 results = {}
105
106 for name, image in test_images.items():
107     # Convert to grayscale if needed
108     if len(image.shape) == 3:
109         gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
110     else:
111         gray = image.copy()
112
113     # Apply different thresholding methods
114     global_binary = apply_global_threshold(gray, 127)
115     otsu_binary = apply_otsu_threshold(gray)
116     adaptive_binary = apply_adaptive_threshold(gray, 11, 2)
117     iterative_binary, T_iter = apply_iterative_threshold(gray)
118
119     # FIX: Set Otsu threshold value as a scalar
120     # cv2.threshold returns (retval, dst)
121     T_otsu_val, _ = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
122
123     # Ensure T_iter is also a scalar
124     T_iter_val = float(T_iter[0]) if isinstance(T_iter, (np.ndarray, list)) else float(T_iter)
125
126     results[name] = {
127         'original': gray,
128         'global': global_binary,
129         'otsu': otsu_binary,
130         'adaptive': adaptive_binary,
131         'iterative': iterative_binary,
132         'T_otsu': float(T_otsu_val), # Convert to float scalar
133         'T_iter': T_iter_val # Convert to float scalar
134     }
135
136 # Visualisasi hasil
137 fig, axes = plt.subplots(len(test_images), 5, figsize=(20, 4*len(test_images)))
138
139 for idx, (name, result) in enumerate(results.items()):
140     # Column 1: Original image - histogram
141     axes[idx, 0].imshow(result['original'], cmap='gray')
142     axes[idx, 0].set_title(f'{name}\nOriginal')
143     axes[idx, 0].axis('off')
144
145     # Column 2-5: Thresholding results
146     methods = ['global', 'otsu', 'adaptive', 'iterative']
147     titles = ['Global (T=127)', f'Otsu (T={result["T_otsu"]:.0f})',
148              'Adaptive', f'Iterative (T={result["T_iter"]:.0f})']
149
150     for col, (method, title) in enumerate(zip(methods, titles), 1):
151         axes[idx, col].imshow(result[method], cmap='gray')
152         axes[idx, col].set_title(title)
153         axes[idx, col].axis('off')
154
155 plt.tight_layout()
156 plt.show()

```

```

1 import numpy as np
2 import cv2
3 import matplotlib.pyplot as plt
4 from skimage import io
5
6 def praktikum_9_1():
7     """
8     Perbandingan teknik thresholding: Global, Otsu, dan Adaptive
9     """
10    print("PRAKTIKUM 9.1: PERBANDINGAN TEKNIK THRESHOLDING")
11    print("-" * 60)
12
13    # Buat citra test dengan berbagai karakteristik
14    def create_test_images():
15        images = {}
16
17        # 1. Citra bimodal (ideal untuk thresholding)
18        img_bimodal = np.zeros((256, 256), dtype=np.uint8)
19        cv2.rectangle(img_bimodal, (80, 80), (220, 220), 50, -1) # Dark object
20        cv2.rectangle(img_bimodal, (180, 180), (220, 220), 200, -1) # Bright object
21        images['Bimodal Image'] = img_bimodal
22
23        # 2. Citra dengan uneven illumination
24        img_uneven = np.zeros((256, 256), dtype=np.uint8)
25        # Create gradient background
26        for i in range(256):
27            img_uneven[i, :] = i // 2
28
29        # Add objects
30        cv2.rectangle(img_uneven, (50, 50), (100, 100), 255, -1)
31        cv2.rectangle(img_uneven, (150, 150), (200, 200), 100, -1)
32        images['Uneven Illumination'] = img_uneven
33
34        # 3. Citra dengan noise
35        img_noisy = np.zeros((256, 256), dtype=np.uint8)
36        cv2.rectangle(img_noisy, (50, 50), (150, 150), 128, -1)
37        # Add Gaussian noise
38        noise = np.random.normal(0, 30, img_noisy.shape)
39        img_noisy = np.clip(img_noisy.astype(float) + noise, 0, 255).astype(np.uint8)
40        images['Noisy Image'] = img_noisy
41
42        # 4. Citra dengan multiple intensity levels
43        img_multi = np.zeros((256, 256), dtype=np.uint8)
44        cv2.rectangle(img_multi, (80, 80), (90, 90), 80, -1) # Dark gray
45        cv2.rectangle(img_multi, (180, 80), (190, 90), 120, -1) # Medium gray
46        cv2.rectangle(img_multi, (170, 80), (180, 90), 180, -1) # Light gray
47        images['Multi-level Image'] = img_multi
48
49    return images
50
51 # Implementasi berbagai metode thresholding
52 def apply_global_threshold(image, T=127):
53     """Global thresholding"""
54     _ , binary = cv2.threshold(image, T, 255, cv2.THRESH_BINARY)
55     return binary
56
57 def apply_otsu_threshold(image):
58     """Otsu's thresholding"""
59     _ , binary = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
60     return binary
61
62 def apply_adaptive_threshold(image, block_size=11, C=2):
63     """Adaptive thresholding"""
64     if len(image.shape) == 3:
65         image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
66     binary = cv2.adaptiveThreshold(image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
67                                  cv2.THRESH_BINARY, block_size, C)
68     return binary
69
70 def apply_iterative_threshold(image, max_iter=100, tolerance=1):
71     """Iterative threshold selection"""
72     # Initialize threshold
73     T = np.mean(image)
74
75     for i in range(max_iter):
76         # Segment image
77         foreground = image[image > T]
78         background = image[image <= T]

```

```

150 # Analisis histogram dan threshold selection
151 print("ANALISIS HISTOGRAM DAN THRESHOLD SELECTION")
152 print("\n * * * ")
153
154 fig, axes = plt.subplots(2, 2, figsize=(12, 18))
155 axes = axes.ravel()
156
157 for idx, (name, result) in enumerate(list(results.items()))[:4]:
158     # Plot histogram
159     hist = cv2.calcHist([result["original"]], [0], None, [256], [0, 256])
160     axes[idx].plot(hist, 'k-', linewidth=2)
161
162     # Add threshold lines
163     axes[idx].axvline(x=t27, color='r', linestyle='--', label=f'Global (t27)', alpha=0.7)
164     axes[idx].axvline(x=result['otsu_t27'], color='g', linestyle='--',
165                      label=f'Otsu (result["{otsu_t27}"]), alpha=0.7)
166     axes[idx].axvline(x=result["l_iter"], color='b', linestyle='--',
167                      label=f'Iterative (result["{l_iter}"]), alpha=0.7)
168
169     axes[idx].set_title(f'{name}(Histogram with Thresholds)')
170     axes[idx].set_xlabel('Intensity')
171     axes[idx].set_ylabel('Frequency')
172     axes[idx].legend()
173     axes[idx].grid(True, alpha=0.3)
174     axes[idx].set_xlim(0, 255)
175
176 plt.tight_layout()
177 plt.show()
178
179 # Evaluasi kuantitatif (simulasi ground truth)
180 print("EVALUASI KUANTITATIF (DEKATAN SIMULASI GROUND TRUTH)")
181 print("\n * * * ")
182
183 # Buat ground truth untuk binomial image
184 gt_binomial = np.zeros((256, 256), dtype=np.uint8)
185 gt_binomial[0:110, 0:110] = 1 # First object
186 gt_binomial[110:220, 110:220] = 1 # Second object
187
188 # Hitung metrics untuk setiap metode
189 def calculate_metrics(binary, ground_truth):
190     """Calculate segmentation metrics"""
191     # Ensure binary images
192     binary = (binary > 0).astype(np.uint8)
193     ground_truth = (ground_truth > 0).astype(np.uint8)
194
195     # True Positive, False Positive, etc.
196     tp = np.sum((binary == 1) & (ground_truth == 1))
197     fp = np.sum((binary == 1) & (ground_truth == 0))
198     fn = np.sum((binary == 0) & (ground_truth == 1))
199     tn = np.sum((binary == 0) & (ground_truth == 0))
200
201     # Calculate metrics
202     accuracy = (tp + tn) / (tp + tn + fp + fn) if (tp+tn+fp+fn) > 0 else 0
203     precision = tp / (tp + fp) if (tp + fp) > 0 else 0
204     recall = tp / (tp + fn) if (tp + fn) > 0 else 0
205     f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
206     iou = tp / (tp + fp + fn) if (tp + fp + fn) > 0 else 0
207
208     return {
209         "accuracy": accuracy,
210         "precision": precision,
211         "recall": recall,
212         "f1_score": f1_score,
213         "iou": iou
214     }
215
216 # Evaluasi untuk binomial image
217 binomial_result = results['binomial_image']
218
219 print(f'Methods: {list(Accuracy:)} {Precision:} {Recall:} {F1-Score:} {IoU}')
220 print("\n * * * ")
221
222 methods = ["global", "otsu", "adaptive", "iterative"]
223 method_names = ["binomial", "Otsu", "Adaptive", "Iterative"]
224
225 for method, method_name in zip(methods, method_names):
226     metrics = calculate_metrics(binomial_result[method], gt_binomial)
227     print(f'Method={method_name} | Accuracy:{metrics["accuracy"]:cib.f3} | Precision:{metrics["precision"]:cib.f3} |'
228           f' F1-score:{metrics["f1_score"]:cib.f3} | Recall:{metrics["recall"]:cib.f3} | IoU:{metrics["iou"]:cib.f3}')

```

```

310 # Stop edge (sharp transition)
311 cv2.rectangle(img, (50, 50), (150, 150), 180, -1)
312 cv2.rectangle(img, (150, 50), (250, 150), 200, -1)
313
314 # Ramp edge (gradual transition)
315 for i in range(100, 150):
316     img[100:240, i] = 50 + (i - 100) * 2
317
318 # Roof edge (triangular)
319 triangle_cnt = np.array([100, 160, (150, 200), (250, 240)])
320 cv2.drawContours(img, [triangle_cnt], 0, 150, -1)
321
322 # Line edge (thin line)
323 cv2.line(img, (50, 200), (150, 250), 200, 3)
324
325 # Add noise into testing robustness
326 noise = np.random.normal(0, 0.5, img.shape)
327 img = np.clip(img.astype(float) + noise, 0, 255).astype(np.uint8)
328
329 return img
330
331 # 2. Implementation of each edge detector
332 def apply_sobel_edge_detection(image, kernel_size=3):
333     sobelx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=kernel_size)
334     sobely = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=kernel_size)
335     magnitude = np.sqrt(sobelx**2 + sobely**2)
336     magnitude = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
337     direction = np.arctan2(sobelx, sobely) * 180 / np.pi
338     return magnitude.astype(np.uint8), direction
339
340 def apply_prewitt_edge_detection(image):
341     kernelx = np.array([-1, 1, 1], [0, 0, 0], [-1, -1, -1])
342     kernely = np.array([-1, 0, 1], [1, -1, 0, 1], [1, 0, 1])
343     prewittx = cv2.filter2D(image.astype(float), -1, kernelx)
344     prewitty = cv2.filter2D(image.astype(float), -1, kernely)
345     magnitude = np.sqrt(prewittx**2 + prewitty**2)
346     magnitude = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
347     return magnitude.astype(np.uint8)
348
349 def apply_canny_edge_detection(image, low_threshold=10, high_threshold=150):
350     edges = cv2.detectMultiScale(image, (5, 5), 1.4)
351     blurred = cv2.GaussianBlur(image, (3, 3), 0)
352     return edges
353
354 def apply_laplacian_edge_detection(image):
355     laplacian = cv2.Laplacian(image, cv2.CV_64F)
356     laplacian = np.abs(laplacian)
357     laplacian = cv2.normalize(laplacian, None, 0, 255, cv2.NORM_MINMAX)
358     return laplacian.astype(np.uint8)
359
360 # 3. Region-based segmentation methods
361 def region_growing_segmentation(image, seeds, threshold=10):
362     segmented = np.zeros_like(image)
363     visited = np.zeros_like(image, dtype=bool)
364     h, w = image.shape
365
366     if isinstance(seeds, tuple):
367         seeds = [seeds]
368
369     for seed in seeds:
370         # Partisiion seed beranda di dalam dimensi gambar (memecah indovisor)
371         if not (0 <= seed[0] < h and 0 <= seed[1] < w) or visited[seed]:
372             continue
373
374         stack = [seed]
375         region_val = float(image[seed]) # simpan nilai awal seed
376
377     while stack:
378         x, y = stack.pop()
379         if visited[x, y]:
380             continue
381
382         visited[x, y] = True
383         segmented[x, y] = 255
384
385     # Check 4-connected neighbors
386     for nx, ny in ((x+1, y), (x-1, y), (x, y+1), (x, y-1)):
387         if 0 <= nx < w and 0 <= ny < h and not visited[nx, ny]:
388             neighbor_val = float(image[nx, ny])
389             # Perbaikan: cast ke float/in untuk mencegah overflow uint8
390             if abs(neighbor_val - region_val) < threshold:
391                 stack.append((nx, ny))

```

```

230 # Visual comparison dengan ground truth
231 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
232
233 # Row 1: Original and Ground Truth
234 axes[0, 0].imshow(bimodal_result['original'], cmap='gray')
235 axes[0, 0].set_title('Original Image')
236 axes[0, 0].axis('off')
237
238 axes[0, 1].imshow(gt_bimodal, cmap='gray')
239 axes[0, 1].set_title('Ground Truth')
240 axes[0, 1].axis('off')
241
242 axes[0, 2].axis('off') # Empty
243
244 # Row 2: Thresholding results dengan overlay errors
245 for idx, (method, method_name) in enumerate(zip(methods[1:], method_names[1:])):
246     result_binary = (bimodal_result[method] > 0).astype(np.uint8)
247
248     # Create error visualization
249     error_image = np.zeros((256, 256, 3), dtype=np.uint8)
250
251     True Positive: white
252     tp_mask = (result_binary == 1) & (gt_bimodal == 1)
253     error_image[tp_mask] = [255, 255, 255]
254
255     False Positive: Red (segmented but not in GT)
256     fp_mask = (result_binary == 1) & (gt_bimodal == 0)
257     error_image[fp_mask] = [255, 0, 0]
258
259     False Negative: Blue (in GT but not segmented)
260     fn_mask = (result_binary == 0) & (gt_bimodal == 1)
261     error_image[fn_mask] = [0, 0, 255]
262
263     axes[1, idx].imshow(error_image)
264     axes[1, idx].set_title(f'{method_name} (Red: FP, Blue: FN)')
265     axes[1, idx].axis('off')
266
267 plt.tight_layout()
268 plt.show()
269
270 # Kesimpulan dan rekomendasi
271 print("HASIL SIMPULAN DAN REKOMENDASI")
272 print("-" * 40)
273 print()
274
275 1. Global Thresholding:
276     - Cocok untuk citra dengan kontras tinggi dan iluminasi uniform
277     - Sensitif terhadap uneven illumination dan noise
278     - Butuh manual threshold selection
279
280 2. Otsu's Method:
281     - Fully automatic
282     - Optimal untuk citra bimodal histogram
283     - Tidak efektif untuk multi-modal atau uneven illumination
284
285 3. Adaptive Thresholding:
286     - Cocok untuk uneven illumination
287     - Robust terhadap variasi intensitas lokal
288     - Parameter block size dan C perlu tuning
289
290 4. Iterative Thresholding:
291     - Self-tuning threshold
292     - Cocok untuk citra dengan distribusi intensity yang jelas
293     - Computationally lebih expensive
294     """
295
296 return results
297
298
299 thresholding_results = praktikum_9_1()
300
301 def praktikum_9_2():
302     """
303     Implementasi edge detection and region-based segmentation
304     """
305     print("""PRAKTIKUM 9.2: EDGE DETECTION DAN REGION-BASED SEGMENTATION""")
306     print("-" * 70)
307
308     # 1. Buat citra test dengan berbagai edge types
309     def create_test_image():
310         """Create test image with different edge types"""
311         # Ukuran citra 300x300
312         img = np.zeros((300, 300), dtype=np.uint8)
313
314

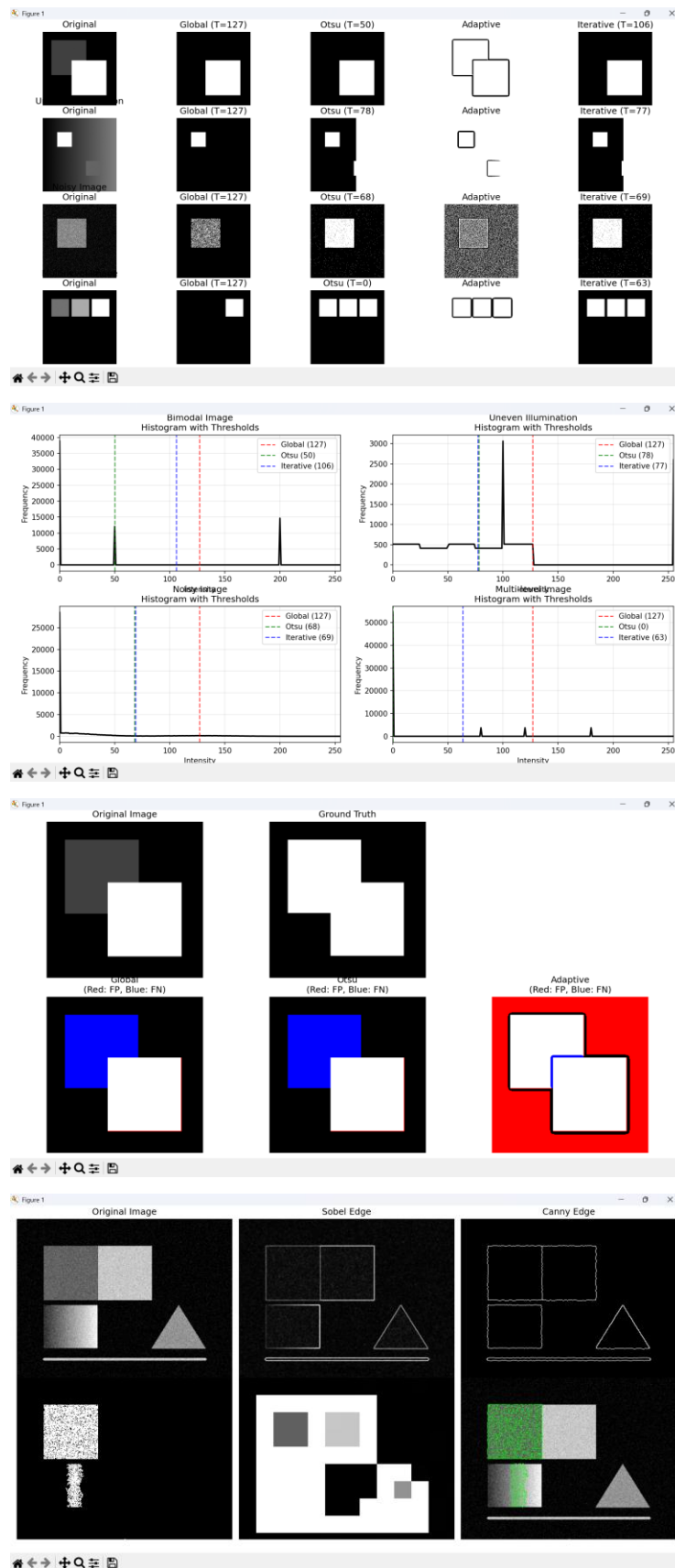
```

```

400     return segmented
401
402 # Simulasi Split & Merge untuk performa lebih kecil
403
404 def region_splitting_merging(img, min_size2, threshold=20):
405     np.random.seed(0)
406     h, w = img.shape
407
408     # Simulasi grid-based splitting sederhana untuk visualisasi
409     for i in range(0, h, min_size):
410         for j in range(0, w, min_size):
411             region = img[i:i+min_size, j:j+min_size]
412             if np.std(region) > threshold:
413                 result[i:i+min_size, j:j+min_size] = np.mean(region)
414
415     result[i:i+min_size, j:j+min_size] = 255
416
417     return result
418
419 # 4. Ekstensi Program
420 test_image = create_edge_test_image()
421 edge_results = {}
422
423 edge_results['Sobel'], sobel_direction = apply_sobel_edge_detection(test_image)
424 edge_results['Prewitt'] = apply_prewitt_edge_detection(test_image)
425 edge_results['Canny'] = apply_canny_edge_detection(test_image)
426 edge_results['Laplacian'] = apply_laplacian_edge_detection(test_image)
427
428 # SEEDS FIX: Index 100 diubah ke 299 agar tidak Out of Bounds
429 seeds = [(79, 100), (200, 100), (200, 200)]
430
431 region_results = {}
432 region_results['Region Growing'] = region_growing_segmentation(test_image, seeds, threshold=25)
433 region_results['Split & Merge'] = region_splitting_merging(test_image, min_size=32, threshold=20)
434
435 # 5. Visualisasi Hasil (FIND INDEXED)
436 print("\nVISUALIZING RESULTS...")
437 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
438
439 # Baris 1: Original dan Edge Detectors
440 axes[0, 0].imshow(test_image, cmap='gray')
441 axes[0, 0].set_title('Original Image')
442
443 # Barisan hanya 2 metode di baris atas agar total kolom pas (3)
444 axes[0, 1].imshow(edge_results['Sobel'], cmap='gray')
445 axes[0, 1].set_title('Sobel Edge')
446
447 axes[0, 2].imshow(edge_results['Canny'], cmap='gray')
448 axes[0, 2].set_title('Canny Edge')
449
450 # Baris 2: Region Based
451 axes[1, 0].imshow(region_results['Region Growing'], cmap='gray')
452 axes[1, 0].set_title('Region Growing')
453
454 axes[1, 1].imshow(region_results['Split & Merge'], cmap='gray')
455 axes[1, 1].set_title('Split & Merge')
456
457 # Overlay Boundaries pada Gambar Original
458 boundaries = cv2.Canny(region_results['Region Growing'], dtype=np.uint8, 50, 150)
459 overlay = cv2.cvtColor(test_image, cv2.COLOR_BGR2RGB)
460 overlay[boundaries > 0] = [0, 255, 0] # Warna hijau
461 axes[1, 2].imshow(overlay, cv2.COLOR_BGR2RGB)
462 axes[1, 2].set_title('Region Boundaries Overlay')
463
464 for ax in axes.ravel():
465     ax.axis('off')
466
467 plt.tight_layout()
468 plt.show()
469
470 print("\nPROSES SELESAI.")
471 return test_image, edge_results, region_results
472
473 # Menjalankan fungsi
474 if __name__ == "__main__":
475     praktikum_9_2()

```

2. Screenshot Output



PRAKTIKUM 9.1: PERBANDINGAN TEKNIK THRESHOLDING

ANALISIS HISTOGRAM DAN THRESHOLD SELECTION

EVALUASI KUANTITATIF (DENGAN SIMULASI GROUND TRUTH)

Method	Accuracy	Precision	Recall	F1-Score	IoU
Global	0.815	0.984	0.548	0.703	0.543
Otsu	0.815	0.984	0.548	0.703	0.543
Adaptive	0.448	0.420	0.985	0.589	0.417
Iterative	0.815	0.984	0.548	0.703	0.543

KESIMPULAN DAN REKOMENDASI

1. Global Thresholding:
 - Cocok untuk citra dengan kontras tinggi dan illumination uniform
 - Sensitif terhadap uneven illumination dan noise
 - Butuh manual threshold selection
2. Otsu's Method:
 - Fully automatic
 - Optimal untuk citra bimodal histogram
 - Tidak efektif untuk multi-modal atau uneven illumination
3. Adaptive Thresholding:
 - Cocok untuk uneven illumination
 - Robust terhadap variasi intensitas lokal
 - Parameter block size dan C perlu tuning
4. Iterative Thresholding:
 - Self-tuning threshold
 - Cocok untuk citra dengan distribusi intensity yang jelas
 - Computationally lebih expensive